

COMPUTER SCIENCE

Unit 1: Algorithm Analysis

*Comprehensive Notes with Detailed Complexity Analysis,
Visual Diagrams & Correctness Proofs*

Table of Contents

1. Insertion Sort

- 1.1 Algorithm Overview
- 1.2 Complexity Analysis
- 1.3 Properties
- 1.4 Visual Walkthrough
- 1.5 Loop Invariant & Correctness

2. Selection Sort

- 2.1 Algorithm Overview
- 2.2 Complexity Analysis
- 2.3 Properties
- 2.4 Visual Walkthrough
- 2.5 Loop Invariant & Correctness

3. Merge Sort

- 3.1 Algorithm Overview
- 3.2 Complexity Analysis
- 3.3 Properties
- 3.4 Visual Walkthrough
- 3.5 Loop Invariant & Correctness

4. Bubble Sort

- 4.1 Algorithm Overview
- 4.2 Complexity Analysis
- 4.3 Properties
- 4.4 Visual Walkthrough
- 4.5 Loop Invariant & Correctness

5. Heapsort

- 5.1 Algorithm Overview
- 5.2 Complexity Analysis
- 5.3 Properties

5.4 Visual Walkthrough

5.5 Loop Invariant & Correctness

6. Quicksort

6.1 Algorithm Overview

6.2 Complexity Analysis

6.3 Properties

6.4 Visual Walkthrough

6.5 Loop Invariant & Correctness

7. Randomized Quicksort

7.1 Algorithm Overview

7.2 Complexity Analysis

7.3 Properties

7.4 Visual Walkthrough

8. Counting Sort

8.1 Algorithm Overview

8.2 Complexity Analysis

8.3 Properties

8.4 Visual Walkthrough

9. Minimum Element

9.1 Algorithm Overview

9.2 Complexity Analysis

9.3 Visual Walkthrough

10. Simultaneous Min-Max

10.1 Algorithm Overview

10.2 Complexity Analysis

10.3 Visual Walkthrough

11. Strassen's Matrix Multiplication

11.1 Algorithm Overview

11.2 Complexity Analysis

11.3 Properties

11.4 Visual Walkthrough

12. Summary

1. Insertion Sort

1.1 Algorithm Overview

Type: Comparison-based, incremental (online) sorting algorithm.

How it works: Builds a sorted subarray one element at a time. Takes one element from the unsorted portion and inserts it into its correct position in the sorted portion by shifting larger elements to the right.

Best Cases to Use

- Small input sizes ($n \leq 20$)
- Nearly sorted arrays (adaptive property)
- Online sorting (data arriving one by one)
- When stability is required with simple implementation

1.2 Complexity Analysis

INSERTION-SORT(A, n)

```

1  for  $i = 2$  to  $n$   $\rightarrow c_1$  ( $n$  times)
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$   $\rightarrow c_2$  ( $n - 1$  times)
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$   $\rightarrow c_4$  ( $n - 1$  times)
6           $A[j + 1] = A[j]$   $\rightarrow c_5$  ( $\sum_{i=2}^n t_i$  times)
 $\rightarrow c_6$  ( $\sum_{i=2}^n (t_i - 1)$  times)
7           $j = j - 1$ 
8       $A[j + 1] = key$   $\rightarrow c_7$  ( $\sum_{i=2}^n (t_i - 1)$  times)
 $\rightarrow c_8$  ( $n - 1$  times)

```

Where t_i = number of times the **while** loop test executes for each i .

Worst Case (Reverse Sorted Array)

In the worst case, each element must move all the way to the front of the sorted subarray. For each i , the inner **while** loop executes i times, so $t_i = i$.

Step-by-Step Derivation:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^n i + c_6 \sum_{i=2}^n (i-1) + c_7 \sum_{i=2}^n (i-1) + c_8$$

Step 1: Evaluate the summations using the arithmetic series formula $\sum_{i=1}^n i = \frac{n(n+1)}{2}$:

$$\sum_{i=2}^n i = \sum_{i=1}^n i - 1 = \frac{n(n+1)}{2} - 1$$

$$\sum_{i=2}^n (i-1) = \sum_{j=1}^{n-1} j = \frac{(n-1)n}{2}$$

Step 2: Substitute back:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \left[\frac{n(n+1)}{2} - 1 \right] + c_6 \left[\frac{n(n-1)}{2} \right] + c_7 \left[\frac{n(n-1)}{2} \right] + c_8$$

Step 3: Expand and collect terms by powers of n :

$$T(n) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n - (c_2 + c_4 + c_5 - c_6 - c_7) + c_8$$

Step 4: Let $a = \frac{c_5+c_6+c_7}{2}$, $b = c_1 + c_2 + c_4 + \frac{c_5-c_6-c_7}{2} + c_8$, $d = c_2 + c_4 + c_5 + c_8$:

$$T(n) = an^2 + bn - d$$

Conclusion: Since $a > 0$ (all $c_i > 0$), the dominant term is n^2 .

$$\boxed{T(n) = \Theta(n^2)}$$

Average Case

On average, each element moves halfway through the sorted subarray. So $t_i \approx i/2$.

The summations become:

$$\sum_{i=2}^n \frac{i}{2} = \frac{1}{2} \left(\frac{n(n+1)}{2} - 1 \right) = \Theta(n^2)$$

$$\sum_{i=2}^n \left(\frac{i}{2} - 1 \right) = \Theta(n^2)$$

Even with half the comparisons, the n^2 term still dominates:

$$T(n) = \Theta(n^2)$$

Best Case (Already Sorted)

When the array is already sorted, $t_i = 1$ for all i (the **while** condition fails immediately each time).

Step-by-Step Derivation:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1)$$

$$T(n) = c_1n + (c_2 + c_4 + c_5 + c_8)(n-1)$$

$$T(n) = c_1n + (c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$

$$T(n) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) = \Theta(n)$$

1.3 Properties

Property	Value	Reasoning
Stable	YES	When $A[j] > key$ (strictly greater), we shift. Equal elements are not shifted past each other. The relative order of equal elements is preserved because we only move when strictly greater.
In-place	YES	Only uses $O(1)$ extra space (variables: key, i, j). All rearrangements happen within the original array. No auxiliary arrays needed.
Adaptive	YES	Running time depends on input order. For nearly sorted arrays, the inner while loop executes fewer times. Best case $\Theta(n)$ when array is already sorted.

1.4 Visual Walkthrough

Sorting $A = [5, 2, 4, 6, 1, 3]$ using Insertion Sort:

1. Insertion Sort

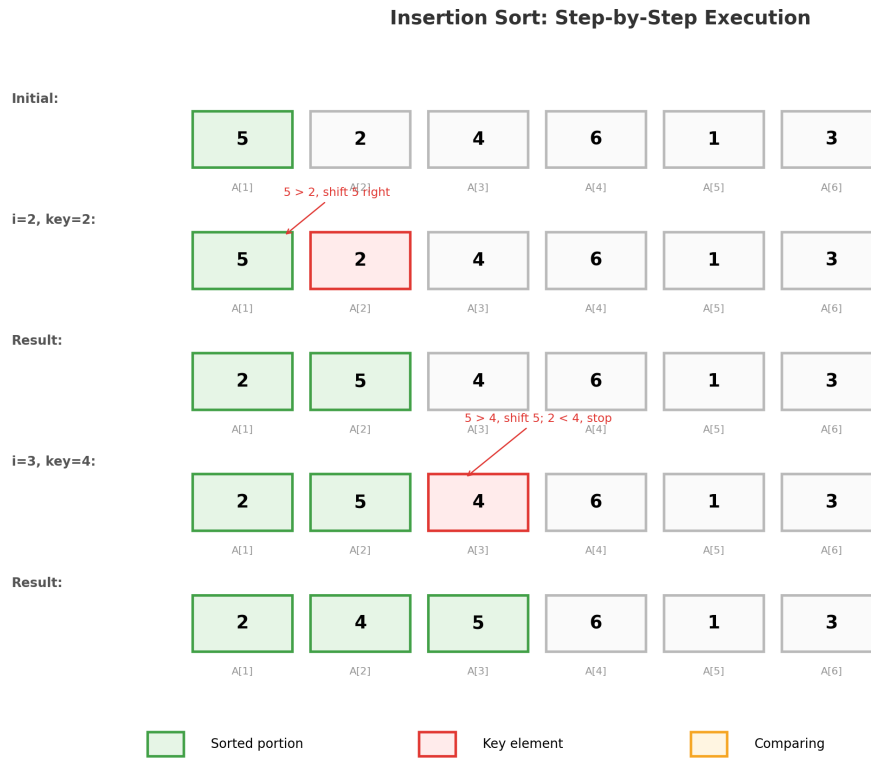


Figure 1: Insertion Sort: Array state at each iteration showing the sorted portion (green), key element being inserted (red), and comparison point (yellow).

1.5 Loop Invariant & Correctness Proof

Loop Invariant (outer for loop)

At the start of each iteration i (where $2 \leq i \leq n + 1$), the subarray $A[1 : i - 1]$ consists of the elements originally in $A[1 : i - 1]$ but in sorted order.

2. Selection Sort

2.1 Algorithm Overview

Type: Comparison-based, selection sorting algorithm.

How it works: Divides the array into sorted and unsorted regions. Repeatedly finds the minimum element from the unsorted region and swaps it with the first element of the unsorted region, expanding the sorted region by one.

Best Cases to Use

- When memory writes are expensive (only $O(n)$ swaps)
- Small datasets where simplicity matters
- When checking all elements is necessary anyway

2.2 Complexity Analysis

SELECTION-SORT(A, n)

1	for $i = 1$ to $n - 1$	$\rightarrow c_1$ (n times)
2	$min_index = i$	
3	for $j = i + 1$ to n	$\rightarrow c_2$ ($n - 1$ times)
4	if $A[j] < A[min_index]$	$\rightarrow c_3$ ($\sum_{i=1}^{n-1} (n - i + 1)$ times)
		$\rightarrow c_4$ ($\sum_{i=1}^{n-1} (n - i)$ times)
5	$min_index = j$	
6	exchange $A[i]$ with $A[min_index]$	$\rightarrow c_5$ ($\sum_{i=1}^{n-1} t_i$ times)
		$\rightarrow c_6$ ($n - 1$ times)

Where t_i = number of times min_index is updated in iteration i .

Worst, Average, and Best Cases

Selection Sort performs the same number of comparisons regardless of input order.

Step-by-Step Derivation:**Step 1:** Evaluate the summation for Line 3:

$$\sum_{i=1}^{n-1} (n - i + 1) = n + (n - 1) + (n - 2) + \dots + 2 = \sum_{k=2}^n k = \frac{n(n+1)}{2} - 1$$

Step 2: Evaluate the summation for Line 4:

$$\sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \dots + 1 = \frac{(n-1)n}{2}$$

Step 3: Combine all terms:

$$T(n) = c_1 n + c_2 (n - 1) + c_3 \left[\frac{n(n+1)}{2} - 1 \right] + c_4 \left[\frac{n(n-1)}{2} \right] + c_5 \cdot O(n) + c_6 (1)$$

Step 4: Expand the n^2 terms:

$$T(n) = \frac{c_3 + c_4}{2} \cdot n^2 + \left(c_1 + c_2 + \frac{c_3}{2} - \frac{c_4}{2} + c_6 \right) n - (c_2 + c_3 + c_6) + c_5 \cdot O(n)$$

Conclusion: The dominant term is n^2 for all cases.

$T(n) = \Theta(n^2)$ for worst, average, and best cases

Note: Even if the array is already sorted, Selection Sort performs all $\frac{n(n-1)}{2}$ comparisons. The number of swaps is $O(n)$ in all cases.

2.3 Properties

Property	Value	Reasoning
Stable	NO	The swap operation can change the relative order of equal elements. Example: $[4_a, 4_b, 3]$ becomes $[3, 4_b, 4_a]$ after first swap. The two 4s have reversed their relative order.
In-place	YES	Uses only $O(1)$ extra space (variables: i, j, min_index). All operations are swaps within the original array.

Adaptive **NO** Running time is $\Theta(n^2)$ regardless of input order. Even if array is already sorted, the algorithm performs all $\frac{n(n-1)}{2}$ comparisons.

2.4 Visual Walkthrough

Sorting $A = [64, 25, 12, 22, 11]$ using Selection Sort:

Iteration	Array State	Action
Initial	[64, 25, 12, 22, 11]	—
$i = 1$	[11 , 25, 12, 22, 64]	Found min = 11 at index 5, swapped with A[1]
$i = 2$	[11, 12 , 25, 22, 64]	Found min = 12 at index 3, swapped with A[2]
$i = 3$	[11, 12, 22 , 25, 64]	Found min = 22 at index 4, swapped with A[3]
$i = 4$	[11, 12, 22, 25 , 64]	Found min = 25 at index 4, swap with itself (no change)
Final	[11, 12, 22, 25, 64]	Sorted

2.5 Loop Invariant & Correctness Proof

Loop Invariant (outer for loop)

At the start of each iteration i (where $1 \leq i \leq n$), the subarray $A[1 : i - 1]$ contains the $i - 1$ smallest elements of the original array in sorted order.

Proof of Correctness:

Initialization: Before first iteration ($i = 1$), $A[1 : 0]$ is empty. Empty subarray trivially contains 0 smallest elements in sorted order. Invariant holds.

Maintenance: Assume invariant holds at start of iteration i ($A[1 : i - 1]$ is sorted). The inner loop finds the minimum in $A[i : n]$. This minimum is the i -th smallest overall element. Swapping it to position i ensures $A[1 : i]$ contains i smallest elements sorted. Incrementing i preserves invariant.

Termination: Loop terminates when $i = n$. By invariant, $A[1 : n - 1]$ contains $n - 1$ smallest elements sorted. The remaining element at $A[n]$ must be the largest. Thus $A[1 : n]$ is sorted. **QED**

3. Merge Sort

3.1 Algorithm Overview

Type: Comparison-based, divide-and-conquer sorting algorithm.

How it works: Recursively divides the array into halves until each subarray has one element, then merges subarrays back together in sorted order.

Best Cases to Use

- Large datasets where $O(n \log n)$ guarantee is needed
- Linked lists (efficient merging without extra space for pointers)
- External sorting (data too large for memory)
- When stability is required with optimal worst-case performance

3.2 Complexity Analysis

MERGE(A, p, q, r)

```

1   $n_L = q - p + 1$   $\rightarrow c_1$ 
2   $n_R = r - q$   $\rightarrow c_2$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays  $\rightarrow c_3$ 
4  for  $i = 0$  to  $n_L - 1$ 
5       $L[i] = A[p + i]$   $\rightarrow c_4$  ( $n_L + 1$  times)
 $\rightarrow c_5$  ( $n_L$  times)
6  for  $j = 0$  to  $n_R - 1$ 
7       $R[j] = A[q + j + 1]$   $\rightarrow c_6$  ( $n_R + 1$  times)
 $\rightarrow c_7$  ( $n_R$  times)
8   $i = 0; j = 0; k = p$ 
12 while  $i < n_L$  and  $j < n_R$   $\rightarrow c_8, c_9, c_{10}$ 
13     if  $L[i] \leq R[j]$   $\rightarrow c_{11}$  (at most  $n_L + n_R + 1$ )
 $\rightarrow c_{12}$  (at most  $n_L + n_R$ )
14          $A[k] = L[i]; i = i + 1$   $\rightarrow c_{13}, c_{14}$ 
16     else  $A[k] = R[j]; j = j + 1$   $\rightarrow c_{15}, c_{16}$ 
18      $k = k + 1$   $\rightarrow c_{17}$ 
20-27 // Copy remaining elements  $\Theta(n_L + n_R) = \Theta(n)$ 

```

MERGE Time: $\Theta(n)$ where $n = r - p + 1$ (total elements merged).

MERGE-SORT(A, p, r)

```

1  if  $p < r$   $\rightarrow c_1$ 
2       $q = \lfloor (p + r) / 2 \rfloor$   $\rightarrow c_2$ 
4      MERGE-SORT(A, p, q)
5      MERGE-SORT(A, q+1, r)  $\rightarrow T(n/2)$ 
 $\rightarrow T(n/2)$ 
7      MERGE(A, p, q, r)  $\rightarrow \Theta(n)$ 

```

Recurrence Relation

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

Solving by Recursion Tree Method

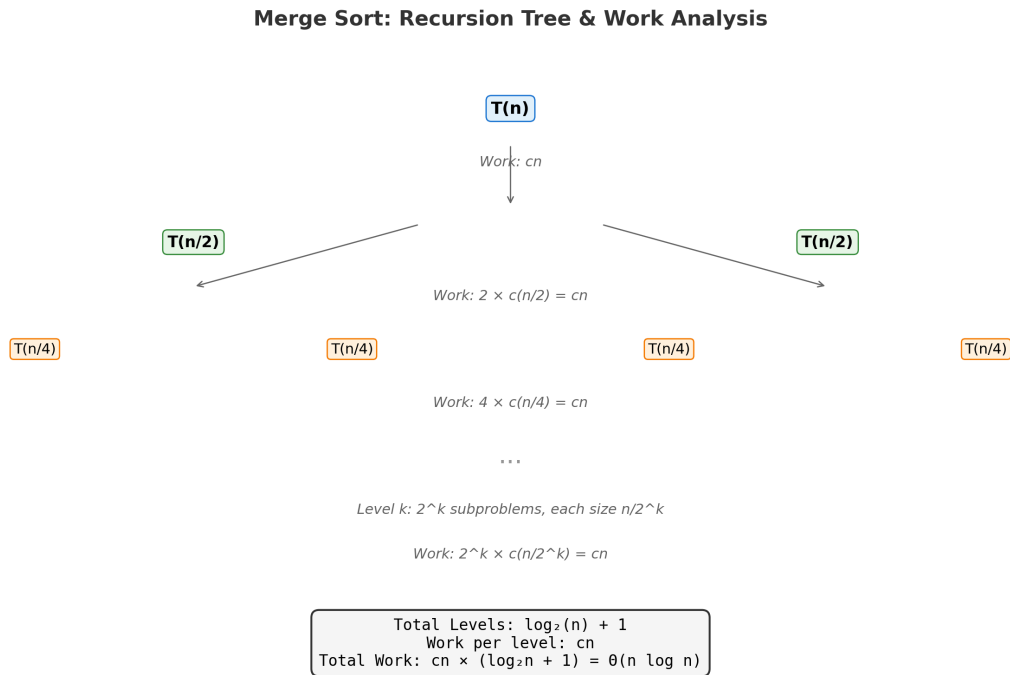


Figure 2: Merge Sort recursion tree showing work at each level. Total work = work per level $\times$ number of levels = $\Theta(n) \times \Theta(\log n) = \Theta(n \log n)$.

Detailed Recursion Tree Analysis:

Level 0 (Root): One problem of size n . Work = cn (the MERGE step).

Level 1: Two problems of size $n/2$. Work = $2 \times c(n/2) = cn$.

Level 2: Four problems of size $n/4$. Work = $4 \times c(n/4) = cn$.

Level k : 2^k problems of size $n/2^k$. Work = $2^k \times c(n/2^k) = cn$.

Number of levels: The recursion stops when $n/2^k = 1$, so $k = \log_2 n$. Total levels = $\log_2 n + 1$.

Total work:

$$T(n) = \sum_{k=0}^{\log_2 n} cn = cn(\log_2 n + 1) = cn \cdot \log_2 n + cn = \Theta(n \log n)$$

Solving by Master Theorem

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

Since $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, **Case 2** of the Master Theorem applies:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(n \log n)$$

This holds for all cases (worst, average, best) since the division and merge structure is identical regardless of input.

3.3 Properties

Property	Value	Reasoning
Stable	YES	In MERGE, when $L[i] \leq R[j]$, we take from L first. Equal elements from the left subarray (which appeared first in the original array) are placed before elements from the right subarray. The $\leq$ ensures stability.
In-place	NO	MERGE requires auxiliary arrays L and R of total size $n_L + n_R = n$. Extra space requirement is $\Theta(n)$.
Adaptive	NO	Always divides into halves and merges regardless of input order. Running time is $\Theta(n \log n)$ even for sorted arrays.

3.4 Visual Walkthrough

Sorting $A = [38, 27, 43, 3, 9, 82, 10]$ using Merge Sort:

Phase	Operation	Result
Divide	Split [38,27,43,3,9,82,10]	[38,27,43] and [3,9,82,10]
Divide	Further split	[38], [27,43], [3,9], [82,10]
Divide	Final split	[38], [27], [43], [3], [9], [82], [10]
Merge	Merge [27] and [43]	[27, 43]
Merge	Merge [38] and [27,43]	[27, 38, 43]
Merge	Merge [3] and [9]	[3, 9]
Merge	Merge [82] and [10]	[10, 82]

Merge	Merge [3,9] and [10,82]	[3, 9, 10, 82]
Merge	Final merge [27,38,43] and [3,9,10,82]	[3, 9, 10, 27, 38, 43, 82]

3.5 Loop Invariant & Correctness Proof

Loop Invariant for MERGE

At the start of each iteration of the main **while** loop, $A[p : k - 1]$ contains the $k - p$ smallest elements of $L[0 : n_L - 1]$ and $R[0 : n_R - 1]$ in sorted order. $L[i]$ and $R[j]$ are the smallest elements of their arrays not yet copied to A .

Proof of MERGE-SORT Correctness (by induction):

Base case: $n = 1$. Array with one element is trivially sorted.

Inductive step: Assume MERGE-SORT works for all sizes $< n$. For size n , it sorts two halves of size $n/2$ correctly (by Inductive Hypothesis), then MERGE correctly combines them. Since MERGE preserves sorted order when combining two sorted arrays, the result is sorted.

Conclusion: MERGE-SORT is correct. **QED**

4. Bubble Sort

4.1 Algorithm Overview

Type: Comparison-based, exchange sorting algorithm.

How it works: Repeatedly steps through the array, compares adjacent elements, and swaps them if they are in the wrong order. Each pass "bubbles" the largest unsorted element to its correct position at the end.

Best Cases to Use

- Educational purposes (simple to understand)
- Nearly sorted arrays with optimization (adaptive variant)
- Small datasets where code simplicity matters

4.2 Complexity Analysis

BUBBLESORT(A, n)

1	for $i = 1$ to $n - 1$	$\rightarrow c_1$ (n times)
2	for $j = n$ downto $i + 1$	
3	if $A[j] < A[j - 1]$	$\rightarrow c_2$ ($\sum_{i=1}^{n-1} (n - i + 1)$ times)
		$\rightarrow c_3$ ($\sum_{i=1}^{n-1} (n - i)$ times)
4	exchange $A[j]$ with $A[j - 1]$	
		$\rightarrow c_4$ ($\sum_{i=1}^{n-1} t_i$ times)

Where t_i = number of swaps in iteration i .

Worst Case (Reverse Sorted)

Every comparison results in a swap. $t_i = n - i$ for all i .

Step-by-Step Derivation:**Step 1:** Evaluate the summation for Line 2:

$$\sum_{i=1}^{n-1} (n - i + 1) = n + (n - 1) + \dots + 2 = \frac{n(n + 1)}{2} - 1$$

Step 2: Evaluate the summation for Line 3 (comparisons):

$$\sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2}$$

Step 3: Evaluate the summation for Line 4 (swaps in worst case):

$$\sum_{i=1}^{n-1} (n - i) = \frac{n(n - 1)}{2}$$

Step 4: Combine all terms:

$$T(n) = c_1 \cdot n + c_2 \left[\frac{n(n + 1)}{2} - 1 \right] + c_3 \left[\frac{n(n - 1)}{2} \right] + c_4 \left[\frac{n(n - 1)}{2} \right]$$

Step 5: Expand and collect by powers of n :

$$T(n) = \frac{c_2 + c_3 + c_4}{2} \cdot n^2 + \left(c_1 + \frac{c_2}{2} - \frac{c_3}{2} - \frac{c_4}{2} \right) n - c_2$$

Conclusion: The dominant term is n^2 .

$$T(n) = \Theta(n^2)$$

Average CaseAbout half the comparisons result in swaps, but the number of comparisons remains $\Theta(n^2)$.

$$T(n) = \Theta(n^2)$$

Best Case (Already Sorted)

No swaps needed, but all comparisons are still performed (without optimization flag).

$$T(n) = c_1 \cdot n + c_2 \left[\frac{n(n+1)}{2} - 1 \right] + c_3 \left[\frac{n(n-1)}{2} \right] = \Theta(n^2)$$

Optimization

With a flag to detect if any swap occurred in a pass, Bubble Sort can achieve $\Theta(n)$ best case by terminating early when the array is already sorted.

4.3 Properties

Property	Value	Reasoning
Stable	YES	We only swap when $A[j] < A[j-1]$ (strictly less). When equal elements are adjacent, no swap occurs. The relative order of equal elements is preserved.
In-place	YES	Only uses $O(1)$ extra space (variables: i, j , temp for swap). All operations are swaps within the original array.
Adaptive	NO	Basic version performs all $\frac{n(n-1)}{2}$ comparisons regardless of input. However, with an optimization flag that stops early if no swaps occur in a pass, it becomes adaptive with best case $\Theta(n)$ for sorted arrays.

4.4 Visual Walkthrough

Sorting $A = [5, 1, 4, 2, 8]$ using Bubble Sort:

Pass	Comparisons	Result After Pass
$i = 1$	(5,1)→swap, (5,4)→swap, (5,2)→swap, (5,8)→no swap	[1, 4, 2, 5, 8]
$i = 2$	(1,4)→no swap, (4,2)→swap, (4,5)→no swap	[1, 2, 4, 5, 8]
$i = 3$	(1,2)→no swap, (2,4)→no swap	[1, 2, 4 , 5, 8]
$i = 4$	(1,2)→no swap	[1, 2, 4, 5, 8] (sorted)

4.5 Loop Invariant & Correctness Proof

Loop Invariant (outer for loop)

At the start of each iteration i (where $1 \leq i \leq n$), the subarray $A[n - i + 2 : n]$ contains the $i - 1$ largest elements of the array in sorted order.

Proof of Correctness:

Initialization: Before first iteration ($i = 1$), $A[n + 1 : n]$ is empty. Empty subarray trivially satisfies the invariant.

Maintenance: During iteration i , the inner loop bubbles the largest element among $A[1 : n - i + 1]$ to position $n - i + 1$. This element is at least as large as all elements in $A[n - i + 2 : n]$. Thus $A[n - i + 1 : n]$ contains i largest elements sorted.

Termination: When $i = n$, $A[2 : n]$ contains $n - 1$ largest elements sorted. $A[1]$ must be the smallest element. Thus entire array is sorted. **QED**

5. Heapsort

5.1 Algorithm Overview

Type: Comparison-based, selection sorting using heap data structure.

How it works: First builds a max-heap from the array. Repeatedly extracts the maximum element (root) and places it at the end of the array, then restores the heap property on the remaining elements.

Best Cases to Use

- When guaranteed $O(n \log n)$ worst-case is needed without extra space
- Priority queue implementations
- Systems with limited memory (in-place)
- When finding k largest/smallest elements

5.2 Complexity Analysis

HEAPSORT(A, n)

1	BUILD-MAX-HEAP(A, n)	
		$\rightarrow \Theta(n)$
2	for $i = n$ downto 2	
3	exchange $A[1]$ with $A[i]$	$\rightarrow c_1 (n - 1 \text{ times})$
		$\rightarrow c_2 (n - 1 \text{ times})$
4	$A.heap\text{-}size = A.heap\text{-}size - 1$	
5	MAX-HEAPIFY(A, 1)	$\rightarrow c_3 (n - 1 \text{ times})$
		$\rightarrow O(\log n) \text{ each, } (n - 1) \text{ times}$

BUILD-MAX-HEAP Analysis

BUILD-MAX-HEAP calls MAX-HEAPIFY on $\lfloor n/2 \rfloor$ nodes, starting from the last non-leaf node and moving upward.

Why BUILD-MAX-HEAP is $\Theta(n)$, not $\Theta(n \log n)$:

At first glance, $n/2$ calls to MAX-HEAPIFY each taking $O(\log n)$ seems like $O(n \log n)$. However, most nodes are at the bottom of the tree where MAX-HEAPIFY is very fast.

Key insight: A node at height h takes $O(h)$ time. The number of nodes at height h is at most $\lceil n/2^{h+1} \rceil$.

$$T(n) = \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right)$$

Using the identity $\sum_{h=0}^{\infty} \frac{h}{2^h} = 2$:

$$T(n) = O(n \cdot 2) = \Theta(n)$$

MAX-HEAPIFY Analysis**MAX-HEAPIFY(A, i)**

Compare $A[i]$ with left and right children
 If violation, swap with larger child and recurse

Time: $O(h)$ where h is the height of the subtree. Since heap height is $\lfloor \log_2 n \rfloor$:

$$T_{\text{heapify}}(n) = O(\log n)$$

Recurrence for MAX-HEAPIFY

In the worst case, the affected subtree can be at most $2n/3$ of the parent (when the last level is exactly half full):

$$T(n) \leq T(2n/3) + \Theta(1)$$

By the Master Theorem: $a = 1$, $b = 3/2$, $f(n) = \Theta(1)$, so $n^{\log_{3/2} 1} = n^0 = 1$. Since $f(n) = \Theta(1) = \Theta(n^{\log_{3/2} 1})$, Case 2 applies:

$$T(n) = \Theta(\log n)$$

Total HEAPSORT Time**Step-by-Step Derivation:**

$$T(n) = T_{\text{build}}(n) + T_{\text{extract}}(n)$$

$$T(n) = \Theta(n) + \sum_{i=2}^n O(\log i)$$

Since $\sum_{i=2}^n \log i = \log(n!) = \Theta(n \log n)$ by Stirling's approximation:

$$T(n) = \Theta(n) + \Theta(n \log n) = \boxed{\Theta(n \log n)}$$

This holds for all cases (worst, average, best) because the heap structure and extraction process are identical regardless of input order.

5.3 Properties

Property	Value	Reasoning
Stable	NO	The swap of $A[1]$ with $A[i]$ can change relative order of equal elements. When the root (max element) is swapped to the end, it may jump over elements equal to it.
In-place	YES	Uses only $O(1)$ extra space (variables for swapping and heap operations). The heap is built within the original array.
Adaptive	NO	Always builds a max-heap and performs $n - 1$ extractions. Running time is $\Theta(n \log n)$ regardless of input order.

5.4 Visual Walkthrough

Sorting $A = [4, 10, 3, 5, 1]$ using Heapsort:

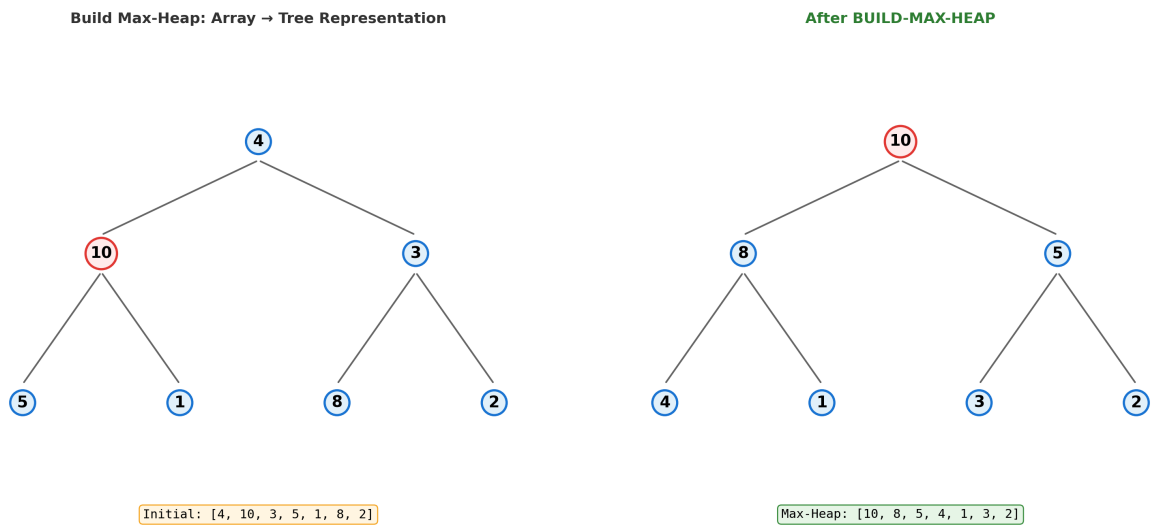


Figure 3: Heapsort: Transformation of array into max-heap. Left: Initial array as complete binary tree. Right: After BUILD-MAX-HEAP, the maximum element (10) is at the root.

Step	Heap Array	Action	Sorted Portion
After Build	$[10, 8, 5, 4, 1, 3, 2]$	Max-heap built	$[]$
$i = 7$	$[2, 8, 5, 4, 1, 3, 10]$	Swap root(10) with $A[7]$	$[10]$
Heapify	$[8, 4, 5, 2, 1, 3]$	Restore heap on 6 elements	$[10]$

$i = 6$	[<u>3</u> , 4, 5, 2, 1, 8]	Swap root(8) with A[6]	[8, 10]
Heapify	[5, 4, 3, 2, 1]	Restore heap on 5 elements	[8, 10]
$i = 5$	[<u>1</u> , 4, 3, 2, 5]	Swap root(5) with A[5]	[5, 8, 10]
...continues until sorted: [1, 2, 3, 4, 5, 8, 10]			

5.5 Loop Invariant & Correctness Proof

Loop Invariant (for HEAPSORT for loop)

At the start of each iteration i (where $2 \leq i \leq n + 1$), the subarray $A[1 : i - 1]$ is a max-heap containing the $i - 1$ smallest elements, and $A[i : n]$ contains the $n - i + 1$ largest elements of the original array in sorted order.

Proof of Correctness:

Initialization: Before first iteration ($i = n$), BUILD-MAX-HEAP ensures $A[1 : n]$ is a max-heap. $A[n + 1 : n]$ is empty. Invariant holds.

Maintenance: At start of iteration i , $A[1 : i]$ is a max-heap. $A[1]$ is the maximum of $A[1 : i]$. Swapping it to $A[i]$ places the $(n - i + 1)$ -th largest element in its final position. Reducing heap-size and calling MAX-HEAPIFY restores the heap property for $A[1 : i - 1]$.

Termination: When $i = 1$, $A[2 : n]$ contains $n - 1$ largest elements sorted. $A[1]$ is the smallest element. Thus $A[1 : n]$ is sorted. **QED**

6. Quicksort

6.1 Algorithm Overview

Type: Comparison-based, divide-and-conquer sorting algorithm.

How it works: Selects a "pivot" element and partitions the array such that elements smaller than pivot are on the left and larger elements are on the right. Recursively sorts the subarrays.

Best Cases to Use

- General-purpose sorting (fastest in practice for most inputs)
- When average-case performance matters more than worst-case
- Cache-friendly applications (good locality of reference)
- When in-place sorting with $O(\log n)$ stack space is acceptable

6.2 Complexity Analysis**PARTITION(A, p, r)**

```

1   $x = A[r]$   $\rightarrow c_1$ 
2   $i = p - 1$   $\rightarrow c_2$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$   $\rightarrow c_3$  ( $n$  times,  $n = r - p + 1$ )
                         $\rightarrow c_4$  ( $n - 1$  times)
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$   $\rightarrow c_5$  ( $t$  times)
7  exchange  $A[i + 1]$  with  $A[r]$   $\rightarrow c_6$  ( $t$  times)
                         $\rightarrow c_7$ 
8  return  $i + 1$   $\rightarrow c_8$ 

```

PARTITION Time: $\Theta(n)$ where $n = r - p + 1$.

QUICKSORT(A, p, r)

```

1  if  $p < r$   $\rightarrow c_1$ 
3       $q = \text{PARTITION}(A, p, r)$ 
4       $\text{QUICKSORT}(A, p, q-1)$   $\rightarrow \Theta(n)$ 
                         $\rightarrow T(k)$ 
5       $\text{QUICKSORT}(A, q+1, r)$   $\rightarrow T(n-k-1)$ 

```

Where $k = q - p$ (number of elements in left partition).

Worst Case (Unbalanced Partitions)

Occurs when the pivot is always the smallest or largest element, creating partitions of size $n - 1$ and 0 .

Step-by-Step Derivation:

Step 1: Set up the recurrence:

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n)$$

Step 2: Unfold the recurrence:

$$T(n) = T(n - 1) + cn$$

$$T(n - 1) = T(n - 2) + c(n - 1)$$

$$T(n - 2) = T(n - 3) + c(n - 2)$$

$$\vdots$$

$$T(2) = T(1) + c(2)$$

Step 3: Substitute back (telescoping):

$$T(n) = T(1) + c(2 + 3 + 4 + \dots + n)$$

$$T(n) = T(1) + c \sum_{i=2}^n i = T(1) + c \left(\frac{n(n+1)}{2} - 1 \right)$$

Step 4: Since $T(1) = \Theta(1)$:

$$T(n) = \Theta(1) + c \left(\frac{n^2 + n - 2}{2} \right) = \Theta(n^2)$$

$$\boxed{T(n) = \Theta(n^2)}$$

Best Case (Balanced Partitions)

Partitions are roughly equal: $k \approx n/2$.

Step-by-Step Derivation:**Step 1:** Set up the recurrence:

$$T(n) = 2T(n/2) + \Theta(n)$$

Step 2: This is the same recurrence as Merge Sort!**Step 3:** Using the Master Theorem: $a = 2, b = 2, f(n) = \Theta(n)$:

$$n^{\log_2 2} = n = f(n) \Rightarrow \text{Case 2 applies}$$

$$T(n) = \Theta(n \log n)$$

Average CaseAssume all splits are equally likely (any partition size k from 0 to $n - 1$ is equally probable):**Expected Time Analysis:**

$$E[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} (E[T(k)] + E[T(n - k - 1)]) + \Theta(n)$$

By symmetry, $E[T(k)] = E[T(n - k - 1)]$:

$$E[T(n)] = \frac{2}{n} \sum_{k=0}^{n-1} E[T(k)] + \Theta(n)$$

Solving via the substitution method (assume $E[T(n)] \leq an \log n + bn$):

$$E[T(n)] \leq \frac{2}{n} \sum_{k=0}^{n-1} (ak \log k + bk) + \Theta(n)$$

Using $\sum_{k=1}^{n-1} k \log k \leq \frac{n^2 \log n}{2} - \frac{n^2}{4}$ for $n \geq 2$:

$$E[T(n)] \leq \frac{2a}{n} \left(\frac{n^2 \log n}{2} - \frac{n^2}{4} \right) + O(n) = an \log n - \frac{an}{2} + O(n)$$

For sufficiently large a , the negative term dominates the $O(n)$:

$$E[T(n)] = O(n \log n)$$

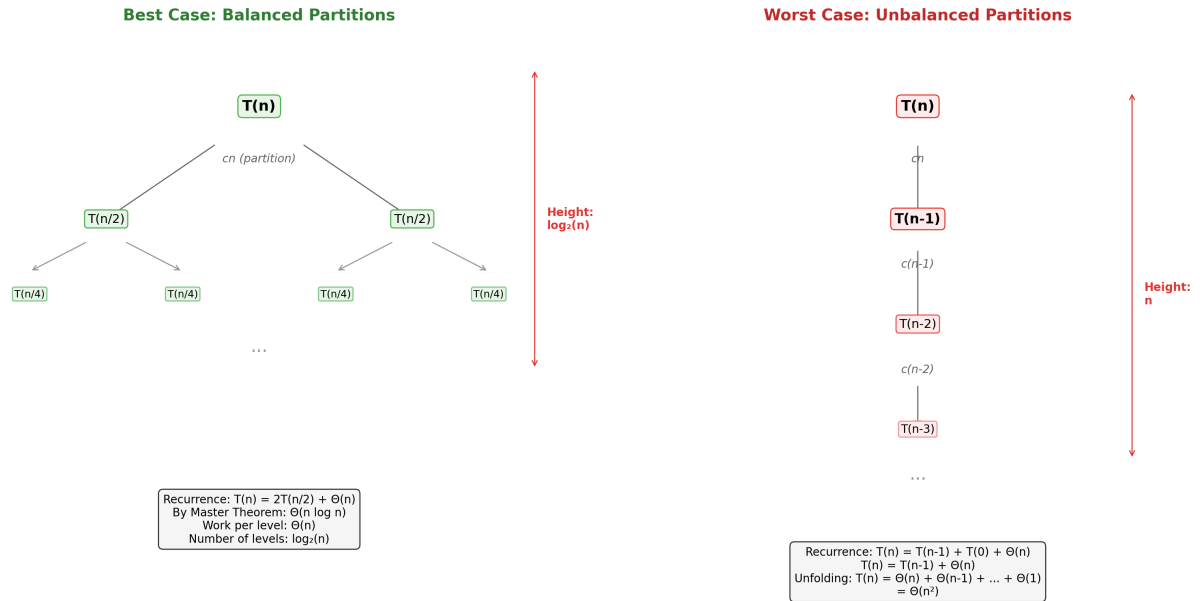


Figure 4: Quicksort recursion trees: Best case (balanced, left) produces a shallow tree with $\Theta(n \log n)$ height. Worst case (unbalanced, right) produces a degenerate tree with $\Theta(n^2)$ height.

6.3 Properties

Property	Value	Reasoning
Stable	NO	The exchange operations in PARTITION can change relative order of equal elements. When $A[j] \leq x$, we swap elements that may jump over equal elements.
In-place	YES	PARTITION rearranges elements within the array using only $O(1)$ extra space. Recursion uses $O(\log n)$ stack space in best/average case, $O(n)$ in worst case.
Adaptive	NO	Running time depends on pivot selection and partition balance, not on input order. Even sorted arrays can cause worst-case behavior with poor pivot choice (e.g., always choosing last element).

6.4 Visual Walkthrough

Sorting $A = [10, 80, 30, 90, 40, 50, 70]$ using Quicksort (Lomuto partition, pivot = last element):

Step	Subarray	Pivot	Partition Result
Initial	[10, 80, 30, 90, 40, 50, 70]	70	[10, 30, 40, 50, 70 , 80, 90]

Left	[10, 30, 40, 50]	50	[10, 30, 40, 50]
Left-Left	[10, 30, 40]	40	[10, 30, 40]
Right	[80, 90]	90	[80, 90]
Final	[10, 30, 40, 50, 70, 80, 90]		

6.5 Loop Invariant & Correctness Proof

Loop Invariant for PARTITION (for loop)

At the start of each iteration j (where $p \leq j \leq r$):

1. $A[p : i]$ contains elements $\leq x$ (pivot)
2. $A[i + 1 : j - 1]$ contains elements $> x$
3. $A[j : r - 1]$ contains elements not yet processed
4. $A[r] = x$ (pivot)

Proof of QUICKSORT Correctness (by induction on subarray size):

Base case: Subarray of 0 or 1 element is trivially sorted.

Inductive step: Assume QUICKSORT works for all subarrays of size $< n$. For a subarray of size n , PARTITION correctly places the pivot at position q such that all elements in $A[p : q - 1]$ are $\leq A[q]$ and all elements in $A[q + 1 : r]$ are $> A[q]$. By the Inductive Hypothesis, the recursive calls correctly sort both partitions. Since pivot is between two sorted partitions with all left elements $\leq$ pivot and all right elements $>$ pivot, the entire subarray is sorted.

Conclusion: QUICKSORT is correct. **QED**

7. Randomized Quicksort

7.1 Algorithm Overview

Type: Comparison-based, randomized divide-and-conquer sorting algorithm.

How it works: Same as Quicksort, but before partitioning, randomly selects an element and swaps it with the last element to use as pivot. This randomization prevents worst-case behavior on specific input patterns (e.g., already sorted arrays).

Best Cases to Use

- When input may be adversarial or nearly sorted
- When worst-case $O(n^2)$ must be avoided with high probability
- General-purpose sorting where average-case guarantee is preferred

7.2 Complexity Analysis

RANDOMIZED-PARTITION(A, p, r)

```

1  i = RANDOM(p, r)
2  exchange A[r] with A[i]
3  return PARTITION(A, p, r)
```

$\rightarrow c_1$
 $\rightarrow c_2$
 $\rightarrow \Theta(n)$

RANDOMIZED-QUICKSORT(A, p, r)

```

1  if p < r
2      q = RANDOMIZED-PARTITION(A, p, r)
3      RANDOMIZED-QUICKSORT(A, p, q-1)
4      RANDOMIZED-QUICKSORT(A, q+1, r)
```

$\rightarrow c_1$
 $\rightarrow \Theta(n)$
 $\rightarrow T(k)$
 $\rightarrow T(n - k - 1)$

Expected Time Analysis

Step-by-Step Derivation of Expected Running Time:

Step 1: The expected running time $E[T(n)]$ satisfies:

$$E[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} (E[T(k)] + E[T(n - k - 1)]) + \Theta(n)$$

Each possible split k (where the pivot ends up at position $k + 1$) occurs with probability $1/n$.

Step 2: By symmetry ($E[T(k)] = E[T(n - k - 1)]$):

$$E[T(n)] = \frac{2}{n} \sum_{k=0}^{n-1} E[T(k)] + \Theta(n)$$

Step 3: Multiply both sides by n :

$$n \cdot E[T(n)] = 2 \sum_{k=0}^{n-1} E[T(k)] + \Theta(n^2)$$

Step 4: Write the same equation for $n - 1$:

$$(n - 1) \cdot E[T(n - 1)] = 2 \sum_{k=0}^{n-2} E[T(k)] + \Theta((n - 1)^2)$$

Step 5: Subtract the two equations:

$$n \cdot E[T(n)] - (n - 1) \cdot E[T(n - 1)] = 2E[T(n - 1)] + \Theta(n)$$

$$n \cdot E[T(n)] = (n + 1) \cdot E[T(n - 1)] + \Theta(n)$$

Step 6: Divide by $n(n + 1)$:

$$\frac{E[T(n)]}{n + 1} = \frac{E[T(n - 1)]}{n} + \frac{\Theta(n)}{n(n + 1)} = \frac{E[T(n - 1)]}{n} + \Theta\left(\frac{1}{n}\right)$$

Step 7: Let $S(n) = \frac{E[T(n)]}{n+1}$. Then $S(n) = S(n - 1) + \Theta(1/n)$:

$$S(n) = \sum_{i=1}^n \Theta\left(\frac{1}{i}\right) = \Theta(H_n) = \Theta(\log n)$$

where H_n is the n -th harmonic number.

Step 8: Substitute back:

$$\frac{E[T(n)]}{n + 1} = \Theta(\log n)$$

$$\boxed{E[T(n)] = \Theta(n \log n)}$$

Worst Case: Still $\Theta(n^2)$, but occurs with very low probability. The probability of consistently picking the worst pivot is $1/n!$ for any specific adversarial input.

7.3 Properties

Property	Value	Reasoning
Stable	NO	Same as Quicksort. The random swap and subsequent PARTITION exchanges can change relative order of equal elements.
In-place	YES	Same as Quicksort. Only $O(1)$ extra space for swapping plus $O(\log n)$ expected stack space for recursion.
Adaptive	NO	Randomization makes running time independent of input order in expectation. No performance improvement for partially sorted input.

7.4 Visual Walkthrough

Sorting $A = [3, 6, 2, 8, 1, 4, 7]$ using Randomized Quicksort:

Call	Subarray	Random Index	Pivot	Partition Result
RQ(A, 1, 7)	[3, 6, 2, 8, 1, 4, 7]	3	2	[2, 6, 3, 8, 1, 4, 7] $\rightarrow$ q=2
RQ(A, 1, 1)	[2]	—	—	Base case
RQ(A, 3, 7)	[6, 3, 8, 1, 4, 7]	5	1	[1, 3, 8, 6, 4, 7] $\rightarrow$ q=3
RQ(A, 4, 7)	[3, 8, 6, 4, 7]	6	7	[3, 6, 4, 7, 8] $\rightarrow$ q=7
...continues recursively until sorted: [1, 2, 3, 4, 6, 7, 8]				

Correctness of Randomized Quicksort:

RANDOMIZED-PARTITION produces a random pivot, but once chosen, PARTITION behaves deterministically. The random choice does not affect the correctness of the partitioning. By the same inductive argument as Quicksort, RANDOMIZED-QUICKSORT correctly sorts any input. The randomization only affects the probability distribution of running times, not correctness. **QED**

8. Counting Sort

8.1 Algorithm Overview

Type: Non-comparison-based, linear time sorting algorithm.

How it works: Counts the number of occurrences of each distinct element, computes cumulative counts to determine positions, then places elements in output array based on these positions.

Best Cases to Use

- When range of input $k = O(n)$
- Integer sorting with small range
- As subroutine in Radix Sort
- When stability is required with linear time

8.2 Complexity Analysis

COUNTING-SORT(A, n, k)

1	let $B[1 : n]$ and $C[0 : k]$ be new arrays	
2	for $i = 0$ to k	$\rightarrow c_1$
		$\rightarrow c_2 \text{ (} k + 1 \text{ times)}$
3	$C[i] = 0$	
		$\rightarrow c_3 \text{ (} k + 1 \text{ times)}$
4	for $j = 1$ to n	
		$\rightarrow c_4 \text{ (} n + 1 \text{ times)}$
5	$C[A[j]] = C[A[j]] + 1$	
		$\rightarrow c_5 \text{ (} n \text{ times)}$
7	for $i = 1$ to k	
		$\rightarrow c_6 \text{ (} k + 1 \text{ times)}$
8	$C[i] = C[i] + C[i - 1]$	
		$\rightarrow c_7 \text{ (} k \text{ times)}$
11	for $j = n$ down to 1	
12	$B[C[A[j]]] = A[j]$	$\rightarrow c_8 \text{ (} n + 1 \text{ times)}$
		$\rightarrow c_9 \text{ (} n \text{ times)}$
13	$C[A[j]] = C[A[j]] - 1$	
		$\rightarrow c_{10} \text{ (} n \text{ times)}$
14	return B	
		$\rightarrow c_{11}$

Time Complexity Derivation

Step-by-Step Derivation:

Step 1: Line 2-3 (initialize count array): $\Theta(k)$

Step 2: Line 4-5 (count occurrences): $\Theta(n)$

Step 3: Line 7-8 (cumulative counts): $\Theta(k)$

Step 4: Line 11-13 (place elements): $\Theta(n)$

Step 5: Total:

$$T(n, k) = \Theta(k) + \Theta(n) + \Theta(k) + \Theta(n) = \Theta(n + k)$$

Step 6: When $k = O(n)$:

$$T(n) = \Theta(n)$$

Space Complexity

Requires two auxiliary arrays: B (output, size n) and C (counts, size $k + 1$):

$$S(n, k) = \Theta(n + k)$$

8.3 Properties

Property	Value	Reasoning
Stable	YES	The algorithm processes A from right to left (line 11: $j = n$ downto 1) and places elements in B using cumulative counts. Elements with equal values appear in B in the same relative order as in A because later occurrences are placed first (due to decrementing $C[A[j]]$).
In-place	NO	Requires two auxiliary arrays: B (output, size n) and C (counts, size $k + 1$). Extra space is $\Theta(n + k)$.
Adaptive	NO	Always performs the same operations regardless of input order. The counting, cumulative sum, and placement steps are executed identically for any input of size n with range k .

8.4 Visual Walkthrough

Sorting $A = [4, 2, 2, 8, 3, 3, 1]$ where $n = 7, k = 8$:

Step	Array C	Description
Initialize	[0, 0, 0, 0, 0, 0, 0, 0]	Indices 0 to 8, all zeros
After counting	[0, 1, 2, 2, 1, 0, 0, 0, 1]	$C[i]$ = count of element i
Cumulative	[0, 1, 3, 5, 6, 6, 6, 6, 7]	$C[i]$ = elements $\leq i$

j	$A[j]$	$C[A[j]]$	Place at $B[.]$	After Decrement	B Array
7	1	1	$B[1] = 1$	$C[1] = 0$	[1, _, _, _, _, _, _]
6	3	5	$B[5] = 3$	$C[3] = 4$	[1, _, _, 3, _, _]
5	3	4	$B[4] = 3$	$C[3] = 3$	[1, _, 3, 3, _, _]
4	8	7	$B[7] = 8$	$C[8] = 6$	[1, _, 3, 3, _, 8]
3	2	3	$B[3] = 2$	$C[2] = 2$	[1, _, 2, 3, 3, _, 8]
2	2	2	$B[2] = 2$	$C[2] = 1$	[1, 2, 2, 3, 3, _, 8]
1	4	6	$B[6] = 4$	$C[4] = 5$	[1, 2, 2, 3, 3, 4, 8]

Loop Invariant for COUNTING-SORT (final for loop)

At the start of each iteration j , B contains the correctly positioned elements from $A[j + 1 : n]$, and C contains the correct remaining counts for placing elements from $A[1 : j]$.

Proof of Correctness:

Initialization: Before first iteration ($j = n$), B is empty. C contains cumulative counts of all elements. Invariant holds vacuously.

Maintenance: At iteration j , $C[A[j]]$ contains the correct final position for $A[j]$ (number of elements $\leq A[j]$ that haven't been placed yet). Placing $A[j]$ at $B[C[A[j]]]$ and decrementing $C[A[j]]$ ensures the next occurrence of the same value goes to the position immediately before. This maintains stability and correct positioning.

Termination: When $j = 0$, all elements from $A[1 : n]$ have been placed in B . By the invariant, all are in correct sorted positions. **QED**

9. Minimum Element

9.1 Algorithm Overview

Type: Selection algorithm, linear search.

How it works: Initializes `min` with the first element, then scans through remaining elements updating `min` whenever a smaller element is found.

Best Cases to Use

- Finding minimum in an unsorted array
- When only minimum is needed (not full sort)
- Any scenario requiring a single linear scan

9.2 Complexity Analysis

MINIMUM(A, n)

1	<i>min</i> = A[1]	→ c_1
2	for <i>i</i> = 2 to <i>n</i>	
		→ c_2 (n times)
3	if <i>min</i> > A[<i>i</i>]	
		→ c_3 ($n - 1$ times)
4	<i>min</i> = A[<i>i</i>]	
		→ c_4 (t times)
5	return <i>min</i>	→ c_5

Where t = number of times a new minimum is found ($0 \leq t \leq n - 1$).

Worst Case (Decreasing Order)

New minimum found at every comparison: $t = n - 1$.

Step-by-Step Derivation:

$$T(n) = c_1 + c_2 \cdot n + c_3(n - 1) + c_4(n - 1) + c_5$$

$$T(n) = c_1 + c_2n + c_3n - c_3 + c_4n - c_4 + c_5$$

$$T(n) = (c_2 + c_3 + c_4) \cdot n + (c_1 - c_3 - c_4 + c_5)$$

$$\boxed{T(n) = \Theta(n)}$$

Average Case

Expected number of updates is the $(n - 1)$ -th harmonic number $H_{n-1} \approx \ln(n)$:

$$T(n) = c_1 + c_2n + c_3(n - 1) + c_4 \cdot H_{n-1} + c_5 = \Theta(n)$$

Best Case (Increasing Order)

No updates needed: $t = 0$.

$$T(n) = c_1 + c_2n + c_3(n - 1) + c_5 = \Theta(n)$$

9.3 Visual Walkthrough

Finding minimum in $A = [45, 12, 78, 23, 67, 8, 34]$:

i	$A[i]$	Comparison	min	Updated?
1	45	—	45	Initial
2	12	$45 > 12$	12	Yes
3	78	$12 > 78$	12	No
4	23	$12 > 23$	12	No
5	67	$12 > 67$	12	No
6	8	$12 > 8$	8	Yes
7	34	$8 > 34$	8	No
Return min = 8				

Loop Invariant

At the start of each iteration i (where $2 \leq i \leq n + 1$), min contains the minimum value among $A[1 : i - 1]$.

Proof of Correctness:

Initialization: Before first iteration ($i = 2$), $min = A[1]$. The subarray $A[1 : 1]$ contains one element, which is trivially the minimum.

Maintenance: Assume min is the minimum of $A[1 : i - 1]$. At iteration i , compare min with $A[i]$. If $A[i] < min$, then $A[i]$ is the new minimum of $A[1 : i]$. Otherwise, min remains the minimum of $A[1 : i]$.

Termination: When $i = n + 1$, min is the minimum of $A[1 : n]$, the entire array. **QED**

10. Simultaneous Min-Max

10.1 Algorithm Overview

Type: Selection algorithm, optimized pairwise comparison.

How it works: Processes elements in pairs, comparing each pair first, then comparing the smaller with current `min` and the larger with current `max`. Reduces total comparisons from $2n - 2$ to approximately $3n/2$.

Best Cases to Use

- When both minimum and maximum are needed simultaneously
- Tournament structures
- Range queries on unsorted data

10.2 Complexity Analysis

SIMULTANEOUS-MIN-MAX(A, n)

1-12	Initialize min, max, and index i	$\Theta(1)$
13	while $i \leq n - 1$	$\rightarrow c_{11} \text{ (} n/2 \text{ times)}$
14	if $A[i] < A[i + 1]$	$\rightarrow c_{12} \text{ (} n/2 - 1 \text{ times)}$
15-18	Compare smaller with min, larger with max	$\rightarrow 2 \text{ comparisons}$
19-23	else (reverse comparison)	$\rightarrow 2 \text{ comparisons}$
24	$i = i + 2$	$\rightarrow c_{21}$

Comparison Count Analysis

Step-by-Step Derivation:

Case 1: n is odd

- Initial setup: 0 comparisons (set $\min = \max = A[1]$)
- Number of pairs in loop: $(n - 1)/2$
- Each pair: 1 comparison (pair vs pair) + up to 2 comparisons (vs min and max) = 3 comparisons
- Total: $3(n - 1)/2 = \lceil 3n/2 \rceil - 2$

Case 2: n is even

- Initial setup: 1 comparison ($A[1]$ vs $A[2]$)
- Number of pairs in loop: $(n - 2)/2$
- Total: $1 + 3(n - 2)/2 = 3n/2 - 2$

In both cases:

$$\text{Total comparisons} = \left\lceil \frac{3n}{2} \right\rceil - 2$$

Since all other operations are $\Theta(1)$ per iteration:

$$T(n) = \Theta(n)$$

Note: Naive approach uses $2n - 2$ comparisons. This saves $n/2 - 2$ comparisons.

Properties

Property	Value	Reasoning
Stable	N/A	Not a sorting algorithm. Stability is not applicable to selection algorithms.
In-place	YES	Uses only $O(1)$ extra space (variables: min, max, i). No auxiliary arrays needed.
Adaptive	NO	Always processes elements in pairs regardless of input order. The number of comparisons is fixed at approximately $3n/2$.

10.3 Visual Walkthrough

Finding min and max in $A = [3, 8, 2, 9, 5, 1, 7, 4]$ ($n = 8$, even):

Step	Action	min	max	Comparisons
Init	$A[1] = 3 < A[2] = 8$	3	8	1
Pair 1	$A[3] = 2 < A[4] = 9 \rightarrow 2 < \text{min? Yes, } 9 > \text{max? Yes}$	2	9	3
Pair 2	$A[5] = 5 > A[6] = 1 \rightarrow 1 < \text{min? Yes, } 5 > \text{max? No}$	1	9	3
Pair 3	$A[7] = 7 > A[8] = 4 \rightarrow 4 < \text{min? No, } 7 > \text{max? No}$	1	9	3

Result: (min, max) = (1, 9) | **Total comparisons:** $10 = 3(8)/2 - 2$

Loop Invariant

At the start of each while loop iteration, min contains the minimum of $A[1 : i - 1]$ and max contains the maximum of $A[1 : i - 1]$.

Proof of Correctness:

Initialization: Before first while iteration, if n is even, we compare $A[1]$ and $A[2]$, setting min and max correctly for $A[1 : 2]$. If n is odd, $\text{min} = \text{max} = A[1]$ for $A[1 : 1]$.

Maintenance: At each iteration, we compare $A[i]$ and $A[i + 1]$. The smaller is compared with min and the larger with max. After processing the pair, min is the minimum of $A[1 : i + 1]$ and max is the maximum of $A[1 : i + 1]$.

Termination: When $i > n - 1$, all elements have been processed. If n is even, all n elements are in pairs. If n is odd, the last element $A[n]$ was included in the initialization. **QED**

11. Strassen's Matrix Multiplication

11.1 Algorithm Overview

Type: Divide-and-conquer matrix multiplication algorithm.

How it works: Recursively divides $n \times n$ matrices into $n/2 \times n/2$ submatrices, then uses 7 clever multiplications (instead of 8 in the standard divide-and-conquer approach) to compute the product, reducing complexity from $O(n^3)$ to $O(n^{2.81})$.

Best Cases to Use

- Large dense matrix multiplication
- When matrix dimensions are powers of 2
- Theoretical applications where lower exponent matters
- When standard $O(n^3)$ is too slow for large matrices

11.2 Complexity Analysis

MATRIX-ADD(A, B) and MATRIX-SUB(A, B)

For $n \times n$ matrices: nested loops over n^2 elements

Time: $\Theta(n^2)$ each

STRASSEN(A, B)

1-5 Base case ($n = 1$): single multiplication

$\rightarrow \Theta(1)$

6-7 Partition A and B into $n/2 \times n/2$ submatrices

$\Theta(1)$ (indexing)

8-17 10 matrix additions/subtractions (S1-S10)

$\rightarrow 10 \times \Theta(n^2/4) = \Theta(n^2)$

18-24 7 recursive multiplications (P1-P7)

$\rightarrow 7 \times T(n/2)$

25-28 8 matrix additions/subtractions for C11-C22

$\rightarrow \Theta(n^2)$

Recurrence Relation

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 7T(n/2) + \Theta(n^2) & \text{if } n > 1 \end{cases}$$

Explanation:

- 7 recursive calls on $n/2 \times n/2$ submatrices (P1 through P7)
- $\Theta(n^2)$ work for additions/subtractions: 10 additions/subtractions of $n/2 \times n/2$ matrices for S-matrices, plus 8 more for combining C submatrices, totaling $\Theta(n^2)$

Solving by Master Theorem

Step-by-Step Derivation:

Given: $T(n) = 7T(n/2) + \Theta(n^2)$

Identify parameters: $a = 7, b = 2, f(n) = \Theta(n^2)$

Step 1: Compute $n^{\log_b a}$:

$$n^{\log_2 7} \approx n^{2.807}$$

Step 2: Compare $f(n)$ with $n^{\log_b a}$:

$$f(n) = \Theta(n^2) = O(n^{\log_2 7 - \epsilon}) \text{ for } \epsilon \approx 0.81$$

Since n^2 is polynomially smaller than $n^{2.807}$, **Case 1** of the Master Theorem applies.

Step 3: Apply Case 1 result:

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 7})$$

$$\boxed{T(n) = \Theta(n^{2.807})}$$

Note: $\log_2(7) \approx 2.807$, which is less than 3 (standard matrix multiplication).

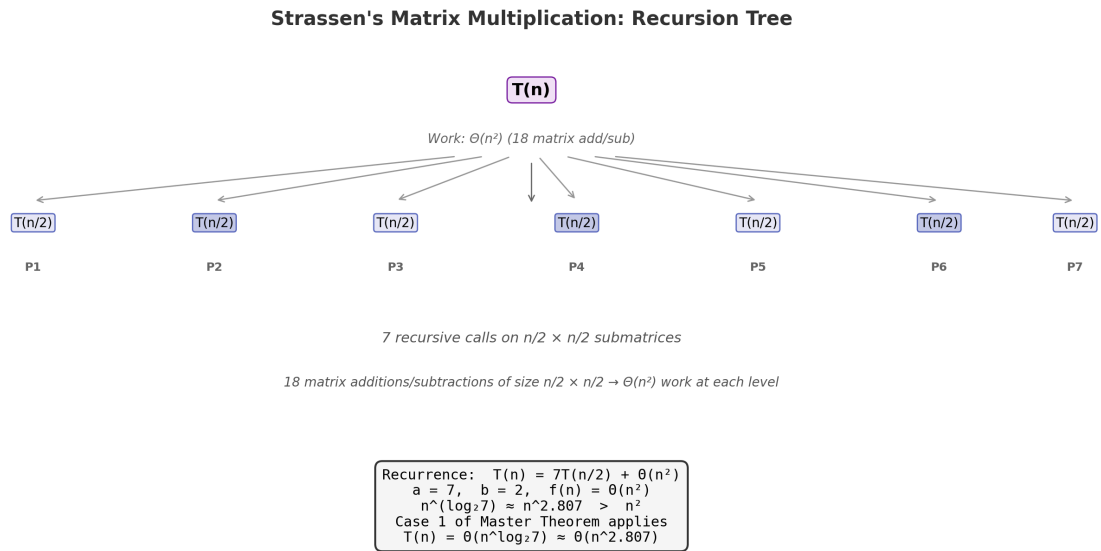


Figure 5: Strassen's algorithm recursion tree: 7 recursive calls on half-sized submatrices, with $\Theta(n^2)$ work at each level for matrix additions/subtractions. Since $7 > 2^2 = 4$, the recursive cost dominates, giving $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

11.3 Properties

Property	Value	Reasoning
Stable	N/A	Not a sorting algorithm. Stability is not applicable to matrix operations.
In-place	NO	Requires multiple auxiliary matrices (S1-S10, P1-P7, and submatrices). Extra space is $\Theta(n^2)$ for the recursive stack and temporary matrices.
Adaptive	NO	Always performs the same 7 recursive multiplications and $\Theta(n^2)$ additions regardless of input matrix values.

11.4 Visual Walkthrough

Computing $C = A \times B$ where $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ ($n = 2$):

Step 1: Partition into submatrices

$$A_{11} = [1], A_{12} = [2], A_{21} = [3], A_{22} = [4]$$

$$B_{11} = [5], B_{12} = [6], B_{21} = [7], B_{22} = [8]$$

Step 2: Compute S matrices (10 additions/subtractions)

S-Matrix	Formula	Value
S_1	$B_{12} - B_{22}$	$[6] - [8] = [-2]$
S_2	$A_{11} + A_{12}$	$[1] + [2] = [3]$
S_3	$A_{21} + A_{22}$	$[3] + [4] = [7]$
S_4	$B_{21} - B_{11}$	$[7] - [5] = [2]$
S_5	$A_{11} + A_{22}$	$[1] + [4] = [5]$
S_6	$B_{11} + B_{22}$	$[5] + [8] = [13]$
S_7	$A_{12} - A_{22}$	$[2] - [4] = [-2]$
S_8	$B_{21} + B_{22}$	$[7] + [8] = [15]$
S_9	$A_{11} - A_{21}$	$[1] - [3] = [-2]$
S_{10}	$B_{11} + B_{12}$	$[5] + [6] = [11]$

Step 3: Compute P matrices (7 recursive multiplications)

P-Matrix	Formula	Value
P_1	$A_{11} \times S_1$	$[1] \times [-2] = [-2]$
P_2	$S_2 \times B_{22}$	$[3] \times [8] = [24]$
P_3	$S_3 \times B_{11}$	$[7] \times [5] = [35]$
P_4	$A_{22} \times S_4$	$[4] \times [2] = [8]$
P_5	$S_5 \times S_6$	$[5] \times [13] = [65]$
P_6	$S_7 \times S_8$	$[-2] \times [15] = [-30]$
P_7	$S_9 \times S_{10}$	$[-2] \times [11] = [-22]$

Step 4: Combine to get C

$$C_{11} = P_5 + P_4 - P_2 + P_6 = [65] + [8] - [24] + [-30] = [19]$$

$$C_{12} = P_1 + P_2 = [-2] + [24] = [22]$$

$$C_{21} = P_3 + P_4 = [35] + [8] = [43]$$

$$C_{22} = P_5 + P_1 - P_3 - P_7 = [65] + [-2] - [35] - [-22] = [50]$$

Verification

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \quad \text{and standard multiplication gives:}$$

$$C_{11} = 1(5) + 2(7) = 19\checkmark \quad C_{12} = 1(6) + 2(8) = 22\checkmark$$

$$C_{21} = 3(5) + 4(7) = 43\checkmark \quad C_{22} = 3(6) + 4(8) = 50\checkmark$$

Correctness Proof (Algebraic Verification)

We show that Strassen's formulas for C_{11} , C_{12} , C_{21} , C_{22} are algebraically equivalent to standard matrix multiplication.

For C_{11} : Standard formula: $C_{11} = A_{11}B_{11} + A_{12}B_{21}$

$$\begin{aligned} C_{11} &= P_5 + P_4 - P_2 + P_6 \\ &= (A_{11} + A_{22})(B_{11} + B_{22}) + A_{22}(B_{21} - B_{11}) - (A_{11} + A_{12})B_{22} + (A_{12} - A_{22})(B_{21} - B_{11}) \end{aligned}$$

Expanding and canceling terms (all cross-terms cancel):

$$\begin{aligned} &= A_{11}B_{11} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22} + A_{22}B_{21} - A_{22}B_{11} - A_{11}B_{22} - A_{12}B_{22} \\ &\quad - A_{12}B_{21} + A_{12}B_{11} + A_{22}B_{21} - A_{22}B_{11} \\ &= A_{11}B_{11} + A_{12}B_{21}\checkmark \end{aligned}$$

Similar algebraic verification holds for C_{12} , C_{21} , and C_{22} . By induction on matrix size (base case $n = 1$ is direct multiplication), STRASSEN is correct. **QED**

12. Summary

Master Theorem Reference

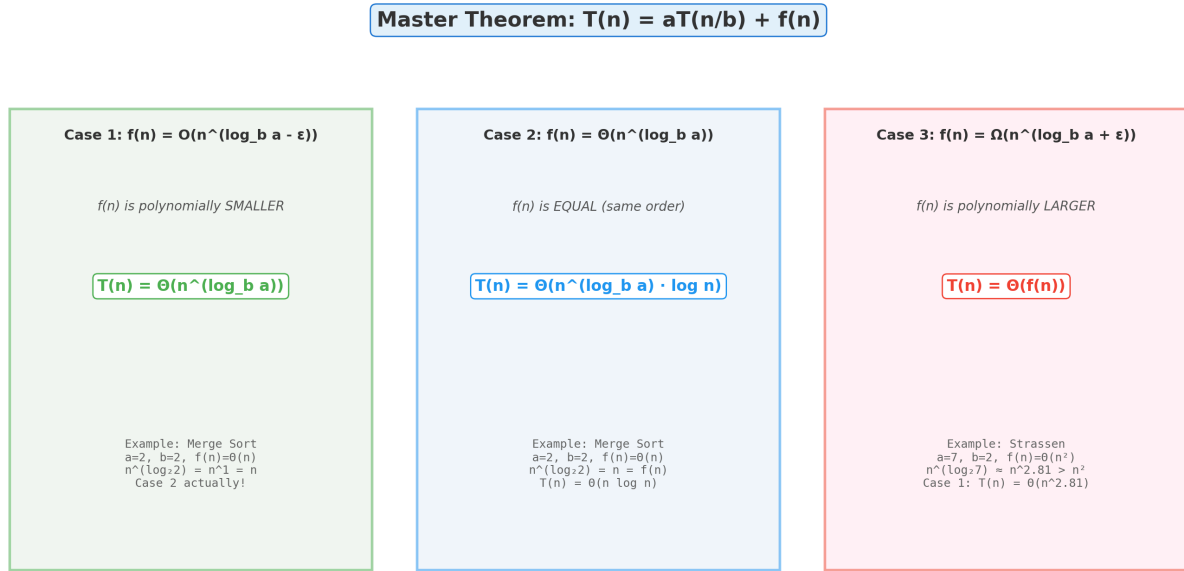


Figure 6: Master Theorem summary: Three cases based on comparing $f(n)$ with $n^{\log_b a}$. Case 1: recursive work dominates. Case 2: balanced. Case 3: non-recursive work dominates.

For recurrences of the form $T(n) = aT(n/b) + f(n)$ where $a \geq 1, b > 1$:

Case	Condition	Result	Examples
Case 1	$f(n) = O(n^{\log_b a - \epsilon})$ for $\epsilon > 0$	$T(n) = \Theta(n^{\log_b a})$	Strassen: $a = 7, b = 2, f(n) = n^2$
Case 2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$	Merge Sort: $a = 2, b = 2, f(n) = n$
Case 3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq cf(n)$	$T(n) = \Theta(f(n))$	Heapify: $a = 1, b = 3/2, f(n) = 1$

Complete Algorithm Comparison

Sorting Algorithms: Complexity & Properties Comparison

Algorithm	Worst Case	Average Case	Best Case	Stable	In-Place
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	No
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Yes	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No	Yes
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	No	Yes
Randomized QuickSort	$O(n^2)^*$	$O(n \log n)$	$O(n \log n)$	No	Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes	No

* Randomized Quicksort: Worst case occurs with very low probability $(1/n!)$ | Bubble Sort best case $O(n)$ with early termination optimization

Figure 7: Comprehensive comparison of all algorithms covered in Unit 1, including their time complexities and key properties.

Key Recurrences and Their Solutions

Algorithm	Recurrence	Solution	Method Used
Merge Sort	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$	Master Theorem (Case 2)
Quicksort (worst)	$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$	Unfolding / Telescoping
Quicksort (best)	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$	Master Theorem (Case 2)
Quicksort (avg)	$E[T(n)] = \frac{2}{n} \sum_{k=0}^{n-1} E[T(k)] + \Theta(n)$	$\Theta(n \log n)$	Substitution Method
Heapify	$T(n) \leq T(2n/3) + \Theta(1)$	$\Theta(\log n)$	Master Theorem (Case 2)
Strassen	$T(n) = 7T(n/2) + \Theta(n^2)$	$\Theta(n^{2.807})$	Master Theorem (Case 1)

Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n)$	Yes	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Yes	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	No	No
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)^*$	Yes	Yes	No*
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No	Yes	No
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	No	Yes	No
Randomized QS	$O(n^2)^{**}$	$O(n \log n)$	$O(n \log n)$	No	Yes	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes	No	No
Minimum	$O(n)$	$O(n)$	$O(n)$	N/A	Yes	No
Sim. Min-Max	$O(n)$	$O(n)$	$O(n)$	N/A	Yes	No
Strassen	$O(n^{2.807})$	$O(n^{2.807})$	$O(n^{2.807})$	N/A	No	No

* Bubble Sort best case $O(n)$ with early termination optimization. Adaptive with optimization flag.

** Randomized Quicksort worst case occurs with very low probability ($1/n!$ for adversarial inputs).

Key Takeaways

When to Use Which Algorithm

- **Small n ($n \leq 20$):** Insertion Sort (low constant factors, adaptive)
- **General purpose, stable:** Merge Sort (guaranteed $O(n \log n)$, stable)
- **General purpose, in-place:** Quicksort or Heapsort
- **Guaranteed $O(n \log n)$ in-place:** Heapsort
- **Integer data, small range $k = O(n)$:** Counting Sort (linear time!)
- **Both min and max needed:** Simultaneous Min-Max ($3n/2$ comparisons vs $2n - 2$)
- **Matrix multiplication:** Strassen for large matrices ($O(n^{2.807})$ vs $O(n^3)$)